Zero to Hero Study Handbook: Real-Time Multimodal Voice Assistant

Module 1: Foundations & Architecture

1.1 What this project does

This repository implements a production-style simulation of a real-time multimodal voice assistant pipeline. The system accepts user utterances, processes them through staged workers (`ingress -> asr -> llm -> tts -> output`), enforces strict latency budgets/timeouts, and applies graceful degradation when latency pressure grows.

Main user-facing entrypoint: - CLI script `rtma-demo` mapped to `realtime_mm.cli:main` in `pyproject.toml`.

Primary use cases in this repo: - Learn real-time pipeline design patterns without external APIs. - Study timeout/fallback behavior under normal and stress latency profiles. - Inspect observability patterns for per-stage and end-to-end latency.

1.2 Core paradigms and design patterns used

Definitions first:

- Asynchronous event-driven pipeline: an architecture where independent worker coroutines process queued work concurrently.
- Backpressure: limiting upstream throughput when downstream is slow, to avoid unbounded queue growth.
- Graceful degradation: reducing output fidelity (for example skipping TTS) to preserve responsiveness.
- Fallback strategy: deterministic backup logic used when primary stage logic times out or errors.
- Data model boundary objects: structured request/result dataclasses passed between stages.

How they are implemented here:

- Async worker pipeline with queues:
  - `RealTimePipeline.run()` creates four `asyncio.Queue` objects and worker tasks for ASR, LLM, TTS, and Output (`src/realtime_mm/pipeline.py`).
- Timeout-wrapped stage execution:
  - `_run_stage()` uses `asyncio.wait_for(...)` and applies fallback logic on timeout/exception.
- Explicit backpressure policy:
  - `_safe_put()` bounds enqueue wait using `enqueue_timeout_ms`; failed enqueue produces `dropped` stage status.
- Adaptive degradation controller:

  - `DegradationController` tracks recent failure ratio and toggles `normal/degraded` mode.
- Typed models:
  - `StageResult`, `RequestEnvelope`, `RequestContext`, `PipelineResult` in `src/realtime_mm/models.py`.

1.3 Architecture: components and interactions

Core components:

- `cli.py`: parses args, builds `PipelineSettings`, runs pipeline, prints budget and runtime summary.
- `config.py`: defines `StageBudget` and `PipelineSettings` (Pydantic settings model).
- `services.py`: simulated primary/fallback implementations for stage behaviors.
- `pipeline.py`: orchestration engine (workers, queueing, timeout/fallback, finalization, degradation control).
- `latency.py`: collects results and renders ASCII metric tables.
- `demo_inputs.py`: deterministic input utterance generator.

ASCII main-flow diagram:

```
User CLI: rtma-demo
      |
      v
main() -> _run_demo(args)
      |
      v
PipelineSettings + RealTimePipeline
      |
      v
for each utterance in run():
  RequestContext(envelope, mode_at_ingress)
      |
      v
  [ingress stage via _run_stage]
      |
      v
  put -> ASR Queue --(worker)--> ASR stage (primary/fallback)
      |
      v
  put -> LLM Queue --(worker)--> LLM stage (primary/fallback)
      |
      v
  put -> TTS Queue --(worker)--> TTS stage or skip (degradation decision)
      |
```

```
      v
  put -> Output Queue --(worker)--> output stage
      |
      v
  _finalize(): PipelineResult + LatencyReport.add() + DegradationController.observe()
      |
      v
render_runtime_summary()
```

Module 2: Repository Map

2.1 Files new contributors should learn first

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`pyproject.toml`	Project metadata, dependencies, script entrypoint, test config	Script: `rtma-demo = realtime_mm.cli:main`	`requires-python >=3.11`; deps: `loguru`, `pydantic`, `pydantic-settings`; dev deps: `pytest`, `pytest-asyncio`
`src/realtime_mm/cli.py`	CLI interface and top-level run orchestration	`_build_parser()`, `_run_demo(args)`, `main()`	CLI flags: `--requests`, `--profile`, `--seed`, `--show-responses`
`src/realtime_mm/config.py`	Runtime settings schema and validation	`StageBudget`, `PipelineSettings`	`env_prefix="RTMA_"`, `env_file=".env"`; latency budgets, queue settings, overload thresholds
`src/realtime_mm/pipeline.py`	Queue-driven stage pipeline and degradation logic	`DegradationController`, `RealTimePipeline`, `run()`, `_run_stage()`, `_safe_put()`, `_finalize()`	`_SENTINEL`; queue `maxsize`; `enqueue_timeout_ms`; `end_to_end_budget_ms`; failure/degradation thresholds
`src/realtime_mm/services.py`	Simulated backend behaviors and fallback generation	`SimulatedServices`, `transcribe_primary/fallback`, `generate_reply_primary/fallback`, `synthesize_primary`, `output_stream`	Random latency/failure/timeout probabilities; `random_seed`

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`src/realtime_mm/models.py`	Data classes for request lifecycle and metrics	`StageResult`, `RequestEnvelope`, `RequestContext`, `PipelineResult`	`StageStatus` literal: `ok/timeout/error/skipped/dropped`; `PipelineMode` literal: `normal/degraded`
`src/realtime_mm/latency.py`	Aggregation and reporting of latency/reliability metrics	`LatencyReport`, `_percentile()`, `_format_table()`	Runtime metric keys: `p50_ms`, `p95_ms`, `timeouts`, `errors`, `skipped`, `dropped`, `deadline_miss_count`
`src/realtime_mm/demo_inputs.py`	Deterministic utterance sourcing for demo runs	`DEFAULT_UTTERANCES`, `get_demo_utterances(count)`	Repeats utterances when requested count exceeds list length
`tests/test_pipeline.py`	Behavioral verification for timeout/degradation/deadline logic	`SlowAsrServices`, `SlowLlmAndTtsServices`, three async test functions	Tests override stage budgets and expect specific degraded reason/status values
`reports/`	Captured run/test artifacts and verification report	`run-normal.txt`, `run-stress.txt`, `pytest.txt`, `verification-report.md`	Real output snapshots used for interpretation

Module 3: Core Execution Flows

3.1 Flow A: CLI entrypoint to pipeline run

Files: - `src/realtime_mm/cli.py` - `src/realtime_mm/config.py` - `src/realtime_mm/pipeline.py`

Step-by-step:

1. `main()` builds parser and parses CLI flags.
2. `main()` executes `asyncio.run(_run_demo(args))`.
3. `_run_demo()` constructs `PipelineSettings(request_count, profile, random_seed)`.
4. `_run_demo()` creates `RealTimePipeline(settings=settings)`.
5. It prints configured budget table via `pipeline.report.render_budget_table(settings)`.

6. It gets utterances through `get_demo_utterances(args.requests)`.
7. It runs `results = await pipeline.run(utterances)`.
8. It optionally prints transcript/reply for each `PipelineResult` when `--show-responses` is enabled.
9. It prints runtime summary using `pipeline.report.render_runtime_summary()`.

Short code fragment:

```
settings = PipelineSettings(
    request_count=args.requests,
    profile=args.profile,
    random_seed=args.seed,
)
pipeline = RealTimePipeline(settings=settings)
utterances = get_demo_utterances(args.requests)
results = await pipeline.run(utterances)
```

Input and output shapes:

- CLI input args:
  - `requests: int`
  - `profile: "normal" | "stress"`
  - `seed: int`
  - `show_responses: bool`
- `_run_demo()` output:
  - returns `None`; side effects are printed tables/logs.
- `pipeline.run(...)` output:
  - `list[PipelineResult]`.

3.2 Flow B: Single-request lifecycle through stage workers

File: - `src/realtime_mm/pipeline.py`

Step-by-step:

1. `run()` creates queues: `asr_queue`, `llm_queue`, `tts_queue`, `output_queue`.
2. `run()` starts workers: `_asr_worker`, `_llm_worker`, `_tts_worker`, `_output_worker`.
3. For each utterance, it creates `RequestContext` containing:
   - `RequestEnvelope(request_id, user_text, created_at)`
   - `mode_at_ingress` from `DegradationController.mode`
4. It executes `ingress` stage via `_run_stage(...)` with `services.ingest_capture`.
5. It enqueues context using `_safe_put(asr_queue, ctx)`.
6. `_asr_worker` runs ASR stage and sets `ctx.transcript`.
7. `_llm_worker` runs LLM stage and sets `ctx.reply_text`.
8. `_tts_worker` either skips TTS (`status="skipped"`) or runs synth stage and sets `ctx.audio_payload`.
9. `_output_worker` runs output stage.

10. `_finalize(ctx)` constructs `PipelineResult`, appends it, records report entry, updates degradation controller, logs one-line status.
11. End-of-stream propagation is done via `_SENTINEL` object moving through all queues.

Short code fragment:

```
ctx = RequestContext(
    envelope=RequestEnvelope(
        request_id=f"req-{idx:03d}",
        user_text=utterance,
        created_at=time.perf_counter(),
    ),
    mode_at_ingress=self._degradation.mode,
)
```

Input and output shapes:

- `RequestEnvelope`:
  - `request_id: str`
  - `user_text: str`
  - `created_at: float`
- `RequestContext` mutable fields added over time:
  - `transcript: str | None`
  - `reply_text: str | None`
  - `audio_payload: bytes | None`
  - `degraded_reasons: list[str]`
  - `stage_results: dict[str, StageResult]`
- `PipelineResult` final fields:
  - `request_id: str`
  - `transcript: str`
  - `reply_text: str`
  - `audio_rendered: bool`
  - `degraded_reasons: list[str]`
  - `stage_results: dict[str, StageResult]`
  - `end_to_end_ms: float`
  - `deadline_met: bool`
  - `mode_at_ingress: "normal" | "degraded"`

3.3 Flow C: Timeout, fallback, and degradation behavior

Files: - `src/realtime_mm/pipeline.py` - `src/realtime_mm/services.py` - `src/realtime_mm/config.py`

Stage execution policy (`_run_stage`):

- Primary callable executes under `asyncio.wait_for(..., timeout=budget.timeout_ms/1000.0)`.
- On timeout:
  - status becomes `"timeout"`

  - reason token appended to `ctx.degraded_reasons`
  - fallback callable executes if provided.
- On exception:
  - status becomes `"error"`
  - reason token appended
  - fallback executes if provided.
- Stage result is always recorded via `_mark_stage(...)`.

Short code fragment:

```
try:
    value = await asyncio.wait_for(primary(), timeout=budget.timeout_ms / 1000.0)
except asyncio.TimeoutError:
    status = "timeout"
    ctx.degraded_reasons.append(timeout_reason)
    if fallback is not None:
        value = fallback()
```

TTS skip policy (`DegradationController.should_skip_tts`):

- Immediately skip when global mode is `degraded`.
- Otherwise skip when either:
  - remaining budget is less than `tts.target_ms + output.target_ms`, or
  - TTS backlog (`in_queue.qsize()`) is at least 3.

Global mode adaptation (`DegradationController.observe`):

- Computes `failed = (not deadline_met) OR any stage in {timeout,error,dropped}`.
- Stores rolling failures in deque of length `overload_window`.
- Switches to `degraded` if failure ratio >= `overload_fail_ratio`.
- Stays degraded for `degrade_cooldown_requests`, then resets to normal.

Drop policy (`_safe_put` + stage marking):

- Queue put uses timeout from `enqueue_timeout_ms`.
- If enqueue fails, downstream stages are marked `dropped` with explicit `detail` strings.

3.4 Flow D: Metrics collection and reporting

File: - `src/realtime_mm/latency.py`

Step-by-step:

1. `_finalize(ctx)` calls `self.report.add(result)`.
2. `LatencyReport.stage_summary()` groups `StageResult` objects per stage and computes:
   - count, `p50_ms`, `p95_ms`, `mean_ms`, `timeouts`, `errors`, `skipped`, `dropped`.
3. `LatencyReport.end_to_end_summary()` computes:

- count, p50_ms, p95_ms, mean_ms, deadline_miss_count, deadline_miss_ratio, degraded_count.

4. render_budget_table(settings) prints configured budgets including computed queueing_slack.
5. render_runtime_summary() prints stage metrics plus:
   - queueing_overhead = max(0, end_to_end_ms - sum(stage latencies))
   - aggregated end-to-end row.

Output shape examples:

- Stage summary record:

```
{
  "count": float,
  "p50_ms": float,
  "p95_ms": float,
  "mean_ms": float,
  "timeouts": float,
  "errors": float,
  "skipped": float,
  "dropped": float,
}
```

- End-to-end summary record:

```
{
  "count": float,
  "p50_ms": float,
  "p95_ms": float,
  "mean_ms": float,
  "deadline_miss_count": float,
  "deadline_miss_ratio": float,
  "degraded_count": float,
}
```

Module 4: Setup & Run Guide

4.1 Environment and dependencies

From pyproject.toml:

- Python: >=3.11
- Runtime deps:
  - loguru>=0.7.2
  - pydantic>=2.8.0
  - pydantic-settings>=2.4.0
- Dev deps:
  - pytest>=8.3.0

  - pytest-asyncio>=0.23.8
- CLI command:
  - rtma-demo = realtime_mm.cli:main

4.2 Install on a clean machine (uv workflow)

```
uv sync --dev
```

4.3 Typical run commands

Normal profile:

```
uv run rtma-demo --requests 8 --profile normal
```

Stress profile with per-request transcript/reply print:

```
uv run rtma-demo --requests 12 --profile stress --show-responses
```

4.4 Configuration via .env and environment variables

`PipelineSettings` uses: - `env_prefix="RTMA_"` - `env_file=".env"` - `extra="ignore"`

There are no mandatory env keys because defaults exist for every setting. Optional override keys come from `PipelineSettings` fields:

- `RTMA_RANDOM_SEED`
- `RTMA_PROFILE`
- `RTMA_REQUEST_COUNT`
- `RTMA_END_TO_END_BUDGET_MS`
- `RTMA_INGRESS`
- `RTMA_ASR`
- `RTMA_LLM`
- `RTMA_TTS`
- `RTMA_OUTPUT`
- `RTMA_QUEUE_MAXSIZE`
- `RTMA_ENQUEUE_TIMEOUT_MS`
- `RTMA_OVERLOAD_WINDOW`
- `RTMA_OVERLOAD_FAIL_RATIO`
- `RTMA_DEGRADE_COOLDOWN_REQUESTS`

Notes: - Stage fields (`ingress/asr/llm/tts/output`) are `StageBudget` models (`target_ms`, `timeout_ms`). - `StageBudget` validation enforces `timeout_ms >= target_ms`. - `overload_fail_ratio` must be in `(0, 1)`.

Example `.env` with safe overrides:

```
RTMA_PROFILE=stress
RTMA_REQUEST_COUNT=12
RTMA_RANDOM_SEED=11
```

```
RTMA_END_TO_END_BUDGET_MS=1500
RTMA_ENQUEUE_TIMEOUT_MS=100
```

4.5 Databases, migrations, and external services

- Database migrations: none present in this repository.
- Seeding steps: none present.
- External providers: none required for the current simulation path.

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan for a new learner

1. `README.md`: understand intended behavior, latency budget philosophy, and tradeoff goals.
2. `pyproject.toml`: identify runtime/development dependencies and CLI entrypoint.
3. `src/realtime_mm/models.py`: learn lifecycle data structures first.
4. `src/realtime_mm/config.py`: map every tunable setting and validation rule.
5. `src/realtime_mm/services.py`: inspect primary and fallback behavior semantics.
6. `src/realtime_mm/pipeline.py`: walk through `run()`, workers, `_run_stage()`, `_finalize()`, and `DegradationController`.
7. `src/realtime_mm/latency.py`: understand how metrics are derived and rendered.
8. `src/realtime_mm/cli.py`: connect user inputs to runtime flow.
9. `tests/test_pipeline.py`: validate your mental model against asserted behaviors.
10. `reports/run-normal.txt` and `reports/run-stress.txt`: map code behavior to observed output.

5.2 Practice exercises (with model answer outlines)

Exercise 1: - Prompt: Trace one request from ingestion to finalization. Which exact functions touch it in order? - Model answer outline: - `RealTimePipeline.run()` creates `RequestContext`. - `run()` executes ingress via `_run_stage(...)`. - Context passes through `_asr_worker()`, `_llm_worker()`, `_tts_worker()`, `_output_worker()`. - Each worker uses `_run_stage(...)` for its stage (except TTS skip branch). - `_finalize(ctx)` creates `PipelineResult`, records metrics, updates degradation state.

Exercise 2: - Prompt: What exact conditions cause TTS to be skipped? - Model answer outline: - In `DegradationController.should_skip_tts(...)`: - return `True` if mode is `degraded`. - else return `True` when `remaining_budget_ms < settings.tts.target_ms + settings.output.target_ms`. - else return `True` when `tts_backlog >= 3`.

Exercise 3: - Prompt: Which fields prove a request degraded, and where are they set? - Model answer outline: - `RequestContext.degraded_reasons` accumulates tokens inside `_run_stage(...)` and queue-drop/skip branches. - Final output includes `PipelineResult.degraded_reasons = sorted(set(ctx.degraded_reasons))` in `_finalize(...)`.

Exercise 4: - Prompt: How is end-to-end deadline miss detected and exposed? - Model answer outline: - `_finalize(...)` computes `end_to_end_ms = _elapsed_ms(created_at)`. - Sets `deadline_met = end_to_end_ms <= settings.end_to_end_budget_ms`. - `LatencyReport.end_to_end_summary()` counts misses using `not r.deadline_met`.

Exercise 5: - Prompt: If enqueue to `llm_queue` times out, which stage statuses are written? - Model answer outline: - In `_asr_worker()`, failed `_safe_put(out_queue, ctx)` appends `llm_queue_drop` reason. - Marks `llm`, `tts`, and `output` as `dropped` with explicit detail strings. - Calls `_finalize(ctx)`.

Exercise 6: - Prompt: What data type and exact field names define per-stage metrics? - Model answer outline: - Type is `StageResult` dataclass. - Fields: `stage`, `status`, `latency_ms`, `target_ms`, `timeout_ms`, `detail`.

Exercise 7: - Prompt: Explain how deterministic behavior is controlled in this simulation. - Model answer outline: - `SimulatedServices.__init__` creates `random.Random(settings.random_seed)`. - Stage latency sampling uses this seeded RNG in `_sample_latency_ms(...)`. - CLI exposes this through `--seed`.

Exercise 8: - Prompt: In stress profile, what changes in latency generation logic compared with normal profile? - Model answer outline: - `SimulatedServices._sample_latency_ms(stage)` chooses an alternate profile dictionary when `settings.profile != "normal"`. - Stress values increase means, jitter, spike probability, and spike magnitudes for each stage.

Understanding Checklist

Use this checklist after studying:

- Can you explain the purpose of each stage (`ingress/asr/llm/tts/output`) and where it is implemented?
- Can you describe exactly how `_run_stage()` handles timeout vs exception vs success?
- Can you explain how queue backpressure is enforced by `_safe_put()` and `enqueue_timeout_ms`?
- Can you describe all states in `StageStatus` and when each is emitted?
- Can you reconstruct how `DegradationController.observe()` transitions into and out of `degraded` mode?
- Can you describe the complete `PipelineResult` structure from memory?
- Can you explain how `queueing_overhead` is computed in `LatencyReport.render_runtime_summary()`?
- Can you identify where CLI flags become `PipelineSettings` fields?

- Can you point to the tests that verify ASR fallback, LLM/TTS degradation, and deadline miss behavior?
- Can you state one architecture tradeoff this code makes and defend why it is reasonable for real-time systems?